

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

Q5. Integrated Deep Learning Model

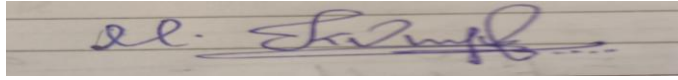
Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

Code of Conduct:

I KUMARAN M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases
--------------------	-----	--	-------------------------	-----------------------------	------------------------------

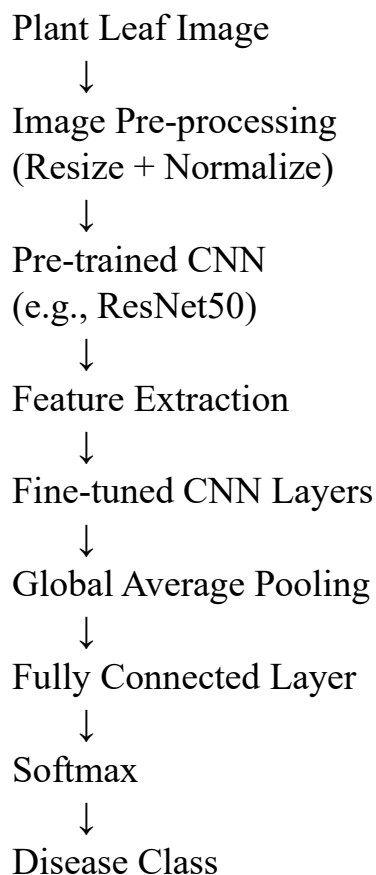
ANSWERS

Q1. Transfer Learning for Plant Disease Detection

Introduction

Plant diseases can significantly reduce crop production. Deep learning-based image classification can automatically identify diseases from plant leaf images. **Transfer learning** is useful because a CNN that has already learned general visual features from a large dataset can be adapted to plant disease classification.

Proposed Framework:



Step 1: Data Collection

Collect images representing different categories such as:

- Healthy leaves
- Fungal diseases
- Bacterial diseases
- Viral diseases
- Different disease stages

The dataset is divided into training, validation, and testing sets.

Step 2: Pre-processing

Images are resized to the input size required by the CNN. Pixel values are normalized to improve training stability.

Step 3: Transfer Learning

A pre-trained model such as **ResNet50**, **EfficientNet**, or **MobileNet** is selected. The original classification layer is removed and replaced with a new classifier for plant disease classes.

Freeze and Fine-tune Strategy

Freeze early layers:

Initial convolutional layers learn general features such as:

- Edges
- Corners
- Basic textures
- Shapes

These features are useful for many image-processing tasks, so they do not need to be relearned.

Fine-tune deeper layers:

The deeper layers learn more task-specific features such as:

- Leaf discoloration
- Spots
- Lesions
- Texture changes
- Disease patterns

Therefore, the last few CNN blocks are unfrozen and trained using a **small learning rate**.

Data Augmentation:

To increase the effective size and diversity of the dataset:

1. **Rotation** – handles leaves captured at different orientations.
2. **Horizontal flipping** – provides additional spatial variations.
3. **Zooming** – helps recognize diseases at different scales.
4. **Random cropping** – makes the model focus on local disease regions.
5. **Brightness adjustment** – handles different lighting conditions.
6. **Contrast adjustment** – improves robustness to image quality variations.
7. **Translation** – handles leaves appearing at different positions.

Advantages:

- Requires less training data.
- Reduces training time.
- Improves feature extraction.
- Usually gives better performance than training from scratch.
- Suitable for datasets with limited plant images.

Conclusion:

Transfer learning provides an efficient approach for plant disease detection. Freezing general feature layers and fine-tuning deeper layers allows the CNN to adapt to plant-specific disease characteristics. Data augmentation further improves robustness and reduces overfitting.

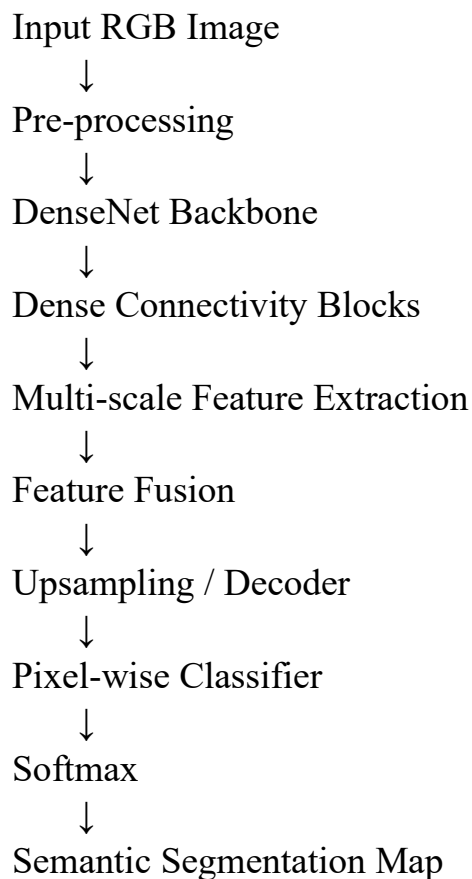
Q2. DenseNet and PixelNet for Autonomous Driving

Introduction:

Autonomous vehicles need to understand their surroundings at the **pixel level**. Semantic segmentation assigns a class to every pixel in an image, such as road, vehicle, pedestrian, building, sky, and traffic sign.

A combination of **DenseNet** and **PixelNet-style pixel-wise prediction** can provide detailed segmentation.

Proposed Architecture:



DenseNet:

DenseNet uses dense connections between layers.

Instead of:

Layer 1 $\rightarrow$ Layer 2 $\rightarrow$ Layer 3

DenseNet uses:

Layer 1 $\rightarrow$ Layer 2

Layer 1 + Layer 2 $\rightarrow$ Layer 3

Layer 1 + Layer 2 + Layer 3 $\rightarrow$ Layer 4

Thus, each layer receives information from all previous layers.

Benefits of Dense Connectivity:

1. Feature reuse:

Earlier features can be reused by later layers.

2. Better gradient flow:

Dense connections provide shorter paths for gradients during backpropagation.

3. Reduced information loss:

Important low-level features can be preserved.

4. Multi-level representation:

The network can use both detailed and high-level semantic features.

Pixel-wise Prediction:

The decoder converts extracted features into a high-resolution feature map. A classifier then assigns a category to each pixel.

For example:

Road → Road

Car → Vehicle

Person → Pedestrian

Building → Building

Sky → Sky

Why Pixel-level Segmentation is Important

Object-level detection only identifies objects using bounding boxes. Semantic segmentation provides much more detailed information about object boundaries and road regions.

This is particularly important for:

- Road detection
- Pedestrian identification
- Vehicle segmentation
- Lane-area understanding
- Obstacle detection

Advantages of Combined Model:

- Strong feature extraction from DenseNet.
- Better feature reuse.
- Detailed pixel-level predictions.
- Better object boundary detection.
- Improved segmentation of small objects.
- Useful for real-time autonomous driving perception.

Conclusion:

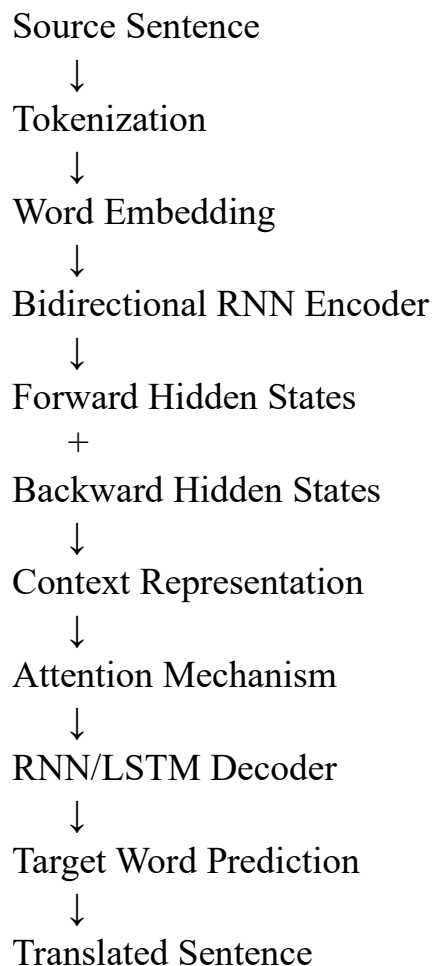
DenseNet provides rich and reusable visual features, while PixelNet-style pixel-wise prediction converts those features into detailed segmentation maps. Their combination can improve the accuracy and reliability of scene understanding in autonomous vehicles.

Q3. Bidirectional RNN Encoder–Decoder for Multilingual Translation

Introduction:

Machine translation converts a sentence from one language into another. An encoder–decoder architecture can learn this transformation automatically. Using a **Bidirectional RNN** in the encoder allows the model to understand information from both directions.

Architecture:



Encoder:

The source sentence is converted into word embeddings.

For example:

I → am → going → home

The Bidirectional RNN processes it in two directions.

Forward direction:

I → am → going → home

Backward direction:

home → going → am → I

The two hidden states are combined to produce a richer representation.

Decoder:

The decoder generates the target sentence one word at a time.

<START>



Target Word 1



Target Word 2



Target Word 3



...



<END>

Role of Attention:

For long sentences, storing the entire sentence in a single context vector can be difficult. Attention allows the decoder to focus on the most relevant source words while generating each target word.

How Bidirectional Context Helps

A word may have different meanings depending on its surrounding words. A Bidirectional RNN sees both:

- Previous context
- Future context

Therefore, it can produce a better representation of the sentence.

Multilingual Extension:

The same framework can be trained using multiple language pairs.

English → French

English → German

English → Hindi

English → Tamil

A language token can also indicate the required target language.

Advantages:

- Captures complete sentence context.
- Improves word representation.
- Handles variable-length sentences.
- Useful for multilingual systems.
- Attention improves long-sentence translation.
- Better contextual understanding than one-directional encoding.

Conclusion:

A Bidirectional RNN encoder combined with an encoder–decoder architecture provides richer contextual information for machine translation. Processing the input in both directions helps the decoder generate more accurate and grammatically meaningful translations.

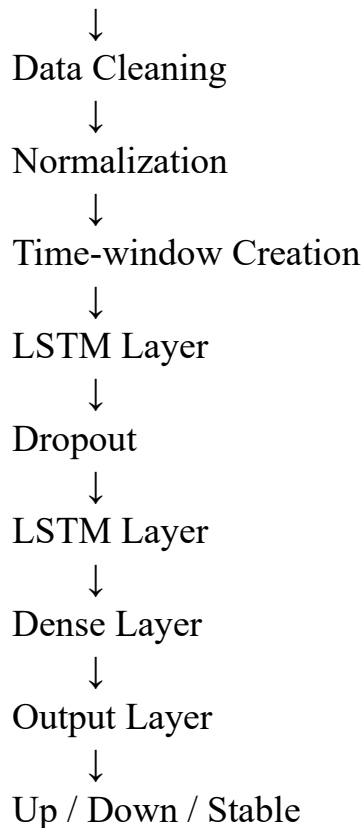
Q4. LSTM and BPTT for Stock Market Trend Prediction

Introduction:

Stock prices are time-dependent and influenced by patterns occurring over different time periods. An LSTM network is suitable for this problem because it can learn long-term dependencies in sequential data.

Proposed Architecture:

Historical Stock Data



Input Data:

Possible input variables include:

- Open price
- Close price
- High price
- Low price
- Trading volume
- Moving averages
- Other technical indicators

A sequence of previous observations is used to predict the future trend.

LSTM Structure:

LSTM contains three important gates:

1. Forget Gate

Determines which previous information should be discarded.

2. Input Gate

Determines which new information should be stored.

3. Output Gate

Determines which information should be passed to the next stage.

The **cell state** carries important information across time steps.

BPTT:

During training, the LSTM is conceptually unfolded across time:

$t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_5$

↓ ↓ ↓ ↓ ↓

LSTM LSTM LSTM LSTM LSTM

The process is:

1. Input historical data sequentially.
2. Generate predictions.
3. Calculate the prediction error.
4. Propagate the error backward through the time steps.
5. Calculate gradients.
6. Update weights using an optimizer.
7. Repeat for multiple epochs.

This process is called **Backpropagation Through Time (BPTT)**.

Why LSTM is Better Than Standard RNN

Standard RNNs may have difficulty preserving information over long sequences because of the **vanishing-gradient problem**.

LSTM uses its cell state and gates to control information flow. Therefore, important historical information can be retained for longer periods.

Advantages:

- Handles sequential data.
- Learns short- and long-term patterns.
- Reduces the effect of vanishing gradients.
- Can use multiple historical features.
- Suitable for time-series prediction.

Important Note:

The model predicts patterns based on historical data; it cannot guarantee future market movements because stock prices are affected by many unpredictable factors.

Conclusion:

An LSTM network trained using BPTT can effectively learn temporal relationships in historical stock data. Its gated memory mechanism makes it more suitable than a standard RNN for long time-series sequences.

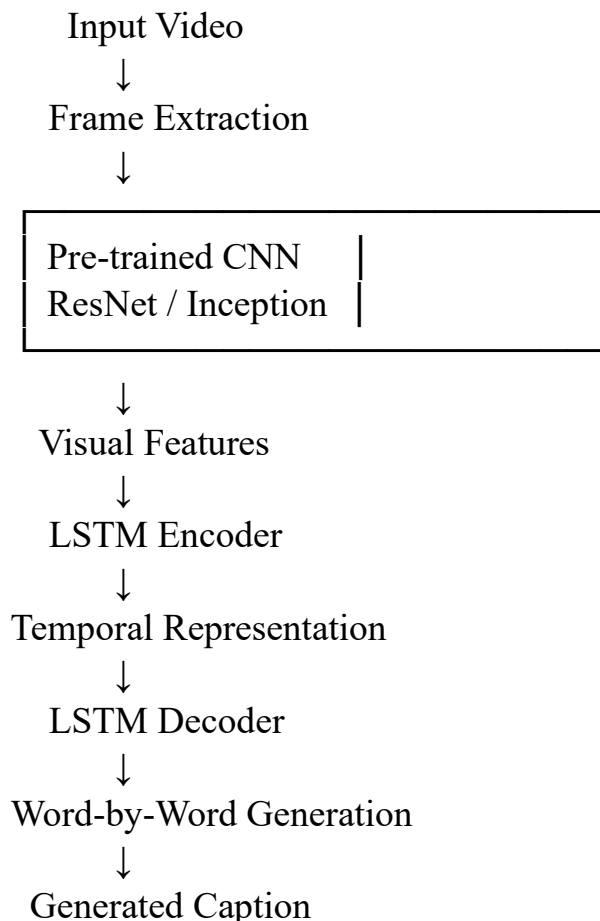
Q5. Integrated Deep Learning Model for Video Caption Generation

Introduction:

Automatic video caption generation converts video content into meaningful natural-language descriptions. The system needs to understand both **visual information** and **temporal changes** in the video.

A suitable solution combines **transfer learning + CNN + LSTM encoder–decoder**.

Overall Architecture:



Step 1: Video Frame Extraction

A video contains many frames. Instead of processing every frame, selected frames can be sampled at regular intervals.

For example:

Frame 1 → Frame 10 → Frame 20 → Frame 30

This reduces computational requirements.

Step 2: Transfer Learning

A pre-trained CNN such as ResNet or Inception extracts meaningful visual features from each frame.

The CNN can identify:

- Objects
- Shapes
- Textures
- People
- Vehicles
- Background elements

The final classification layer is removed and the CNN is used as a feature extractor.

Step 3: LSTM Encoder

The feature vectors from consecutive frames are given to an LSTM.

For example:

Frame 1 feature → LSTM

Frame 2 feature → LSTM

Frame 3 feature → LSTM

Frame 4 feature → LSTM

The LSTM learns how the visual content changes over time.

Step 4: LSTM Decoder

The decoder converts the learned representation into language.

It begins with a start token and predicts one word at a time.

<START>

↓

A

↓

man

↓

is

↓

riding

↓

a

↓

bicycle

↓

<END>

Step 5: Word Prediction

At each step, the decoder calculates probabilities for possible next words. The word with the highest suitable probability is selected.

Role of Each Component

Component	Main Function
Video frames	Provide visual input
Pre-trained CNN	Extract visual features
Transfer learning	Reuse previously learned features
LSTM Encoder	Learn temporal relationships
LSTM Decoder	Generate natural language
Softmax	Predict next word
Attention	Focus on important frames/features

Role of Attention:

Attention can improve caption generation by allowing the decoder to focus on relevant video frames while producing particular words.

For example, when generating a word related to an action, the model can focus more strongly on frames where that action occurs.

Advantages:

- Uses knowledge from pre-trained CNNs.
- Requires less training than training the CNN from scratch.
- Captures temporal information using LSTM.
- Generates captions sequentially.
- Can describe actions and events.
- Attention can improve contextual understanding.

Conclusion:

The integrated model combines the strengths of different deep learning techniques. **Transfer learning and CNNs understand visual content**, while the **LSTM encoder learns temporal information** and the **LSTM decoder converts this information into meaningful sentences**. Therefore, the complete architecture can automatically generate captions describing events in videos.